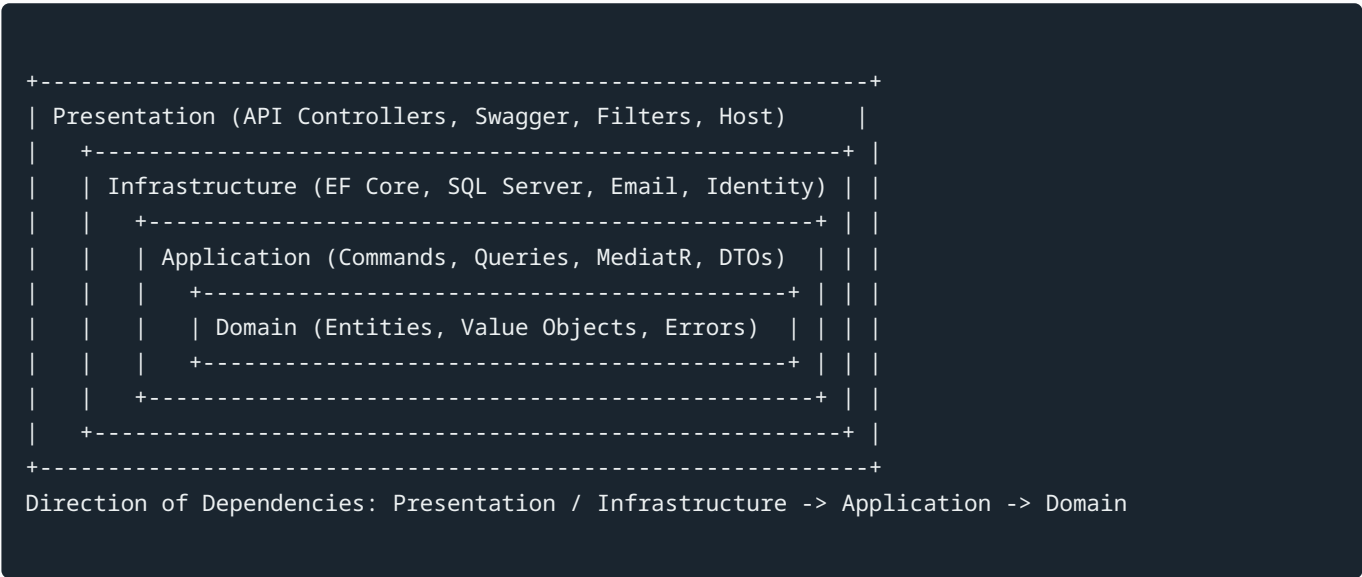


CQRS in Clean Architecture with MediatR

Architectural Foundations, Pipeline Behaviors, and Production Implementation in ASP.NET Core

1. Clean Architecture: Layer Boundaries & Dependency Rule

Clean Architecture structures software systems into concentric layers where dependencies strictly point inward. Outer layers depend on inner layers; inner layers have zero dependencies on outer layers or external frameworks.



Responsibilities by Layer

- **Domain:** Contains enterprise business rules, domain entities, value objects, domain events, and domain-level exceptions. Has no third-party dependencies (not even EF Core).
- **Application:** Implements use cases. Coordinates domain entities and infrastructure abstractions. Contains Commands, Queries, Request Handlers, DTOs, Validators, and Pipeline Behaviors. Depends only on Domain.
- **Infrastructure:** Implements interfaces defined in the Application layer (e.g., database contexts, external payment gateways, token generation). Depends on Application and Domain.
- **Presentation (Web API):** Entry point for HTTP requests. Contains Controllers or Minimal API route endpoints. Responsible only for HTTP serialization, status codes, and routing requests to the Application layer.

2. CQRS (Command Query Responsibility Segregation)

CQRS is an architectural pattern originated by Greg Young. It states that the data model used for modifying data (Commands) should be separated from the data model used for reading data (Queries).

Commands (Write Operations)	Queries (Read Operations)
-----------------------------	---------------------------

- Represent intent to alter state (e.g., `CreateVehicleCommand`).
- Mutate data and enforce domain validation.
- Typically return an identifier, a result status, or `void` .
- Must execute through transactional domain models to ensure consistency.

- Represent intent to retrieve data (e.g., `GetVehicleByIdQuery`).
- Never modify state (Safe & Idempotent).
- Return read-optimized DTOs directly.
- Bypass domain entities and change tracking for performance.

Why CQRS is Preferred: In traditional architectures, a single model (often an EF Core entity) is forced to serve both reading and writing. As applications grow, read queries require aggregations and projections across multiple tables, while write commands require domain validation and invariant checks. Separating them decouples optimization strategies: write pipelines focus on transactional correctness, while read pipelines focus on raw query speed.

3. The Role of MediatR

MediatR is an in-process implementation of the Mediator pattern. It decouples the sender of a message (e.g., an API Controller) from the receiver/handler that processes it.

Key MediatR Abstractions

- **`IRequest<TResponse>`** : Marker interface defining a message (Command or Query) that produces a single response of type `TResponse` .
- **`IRequestHandler<TRequest, TResponse>`** : Defines the handler that executes the specific business logic for `TRequest` .
- **`ISender`** : Specialized interface used by Controllers to dispatch a single request without exposing notification publishing methods.
- **`IPipelineBehavior<TRequest, TResponse>`** : Interceptor pattern for cross-cutting concerns (validation, logging, transactions, metric tracking) surrounding handler execution.

4. Step-by-Step Implementation

Step 1: Domain Entity

```
namespace Domain.Entities;

public sealed class Vehicle
{
    public int Id { get; private set; }
    public string Vin { get; private set; }
    public string Model { get; private set; }
    public bool IsActive { get; private set; }

    private Vehicle() { } // Required by EF Core

    public Vehicle(string vin, string model)
    {
        Vin = !string.IsNullOrEmpty(vin) ? vin : throw new ArgumentException("VIN cannot be empty.");
        Model = !string.IsNullOrEmpty(model) ? model : throw new ArgumentException("Model cannot be empty.");
        IsActive = true;
    }

    public void Deactivate() => IsActive = false;
}
```

Step 2: Command & Handler (Write Path)

```
using MediatR;
using Domain.Entities;

namespace Application.Vehicles.Commands.CreateVehicle;

// Command Contract
public sealed record CreateVehicleCommand(string Vin, string Model) : IRequest;

// Command Handler
public sealed class CreateVehicleCommandHandler : IRequestHandler
{
    private readonly IApplicationDbContext _context;

    public CreateVehicleCommandHandler(IApplicationDbContext context)
    {
        _context = context;
    }

    public async Task Handle(CreateVehicleCommand request, CancellationToken
cancellationToken)
    {
        var vehicle = new Vehicle(request.Vin, request.Model);

        _context.Vehicles.Add(vehicle);
        await _context.SaveChangesAsync(cancellationToken);

        return vehicle.Id;
    }
}
```

Step 3: Query & Handler (Read Path - Optimized)

```
using MediatR;
using Microsoft.EntityFrameworkCore;

namespace Application.Vehicles.Queries.GetVehicleById;

public sealed record VehicleResponse(int Id, string Vin, string Model, bool IsActive);

public sealed record GetVehicleByIdQuery(int Id) : IRequest;

public sealed class GetVehicleByIdQueryHandler : IRequestHandler
{
    private readonly IApplicationDbContext _context;

    public GetVehicleByIdQueryHandler(IApplicationDbContext context)
    {
        _context = context;
    }

    public async Task Handle(GetVehicleByIdQuery request, CancellationToken
cancellationTokentoken)
    {
        // Read optimization: AsNoTracking + direct database-level projection to DTO
        return await _context.Vehicles
            .AsNoTracking()
            .Where(v => v.Id == request.Id)
            .Select(v => new VehicleResponse(v.Id, v.Vin, v.Model, v.IsActive))
            .FirstOrDefaultAsync(cancellationTokentoken);
    }
}
```

Step 4: Cross-Cutting Pipeline Behavior (Validation)

Using MediatR pipeline behaviors, FluentValidation executes automatically before the request reaches the handler.

```

using MediatR;
using FluentValidation;

namespace Application.Common.Behaviors;

public sealed class ValidationBehavior : IPipelineBehavior
    where TRequest : IRequest
{
    private readonly IEnumerable< IValidator> _validators;

    public ValidationBehavior(IEnumerable< IValidator> validators)
    {
        _validators = validators;
    }

    public async Task Handle(
        TRequest request,
        RequestHandlerDelegate next,
        CancellationToken cancellationToken)
    {
        if (!_validators.Any())
            return await next();

        var context = new ValidationContext(request);
        var validationResults = await Task.WhenAll(
            _validators.Select(v => v.ValidateAsync(context, cancellationToken)));

        var failures = validationResults
            .SelectMany(r => r.Errors)
            .Where(f => f != null)
            .ToList();

        if (failures.Count != 0)
            throw new ValidationException(failures);

        return await next();
    }
}

```

Step 5: Registration in Program.cs

```
using MediatR;
using FluentValidation;
using Application.Common.Behaviors;

var builder = WebApplication.CreateBuilder(args);

// Register MediatR handlers from the Application assembly
builder.Services.AddMediatR(cfg => {
    cfg.RegisterServicesFromAssembly(typeof(CreateVehicleCommand).Assembly);
    cfg.AddBehavior(typeof(IPipelineBehavior<, >), typeof(ValidationBehavior<, >));
});

// Register all FluentValidation validators in the assembly
builder.Services.AddValidatorsFromAssembly(typeof(CreateVehicleCommand).Assembly);

builder.Services.AddControllers();
var app = builder.Build();
app.MapControllers();
app.Run();
```

Step 6: Thin Controller Implementation

Controllers do not contain business logic or database access. They only accept inputs and dispatch requests through `ISender`.

```
[ApiController]
[Route("api/[controller]")]
public class VehiclesController : ControllerBase
{
    private readonly ISender _sender;

    public VehiclesController(ISender sender)
    {
        _sender = sender;
    }

    [HttpPost]
    public async Task Create([FromBody] CreateVehicleCommand command, CancellationToken ct)
    {
        var id = await _sender.Send(command, ct);
        return CreatedAtAction(nameof(GetById), new { id }, id);
    }

    [HttpGet("{id:int}")]
    public async Task GetById(int id, CancellationToken ct)
    {
        var result = await _sender.Send(new GetVehicleByIdQuery(id), ct);
        return result is null ? NotFound() : Ok(result);
    }
}
```

5. Technical Trade-offs & Critical Notes

- **In-Memory Mediator vs. Distributed Message Broker:** MediatR is strictly an *in-process* messaging system. It does not replace message queues (like RabbitMQ or Azure Service Bus) for asynchronous cross-service communication.
- **Over-Engineering Risk:** For trivial CRUD applications with no complex business rules, wrapping simple EF Core calls in separate Command/Query/Handler classes introduces class proliferation without clear architectural gain. CQRS with MediatR provides substantial value as business rules, domain events, and pipeline concerns increase in complexity.
- **Vertical Slice Architecture Alternative:** In large solutions, organizing folders by technical type (`Controllers/` , `Services/` , `Repositories/`) slows feature velocity. Grouping by feature (e.g., `Features/Vehicles/CreateVehicle/` containing Command, Handler, Validator, and DTO in one place) keeps related changes localized.